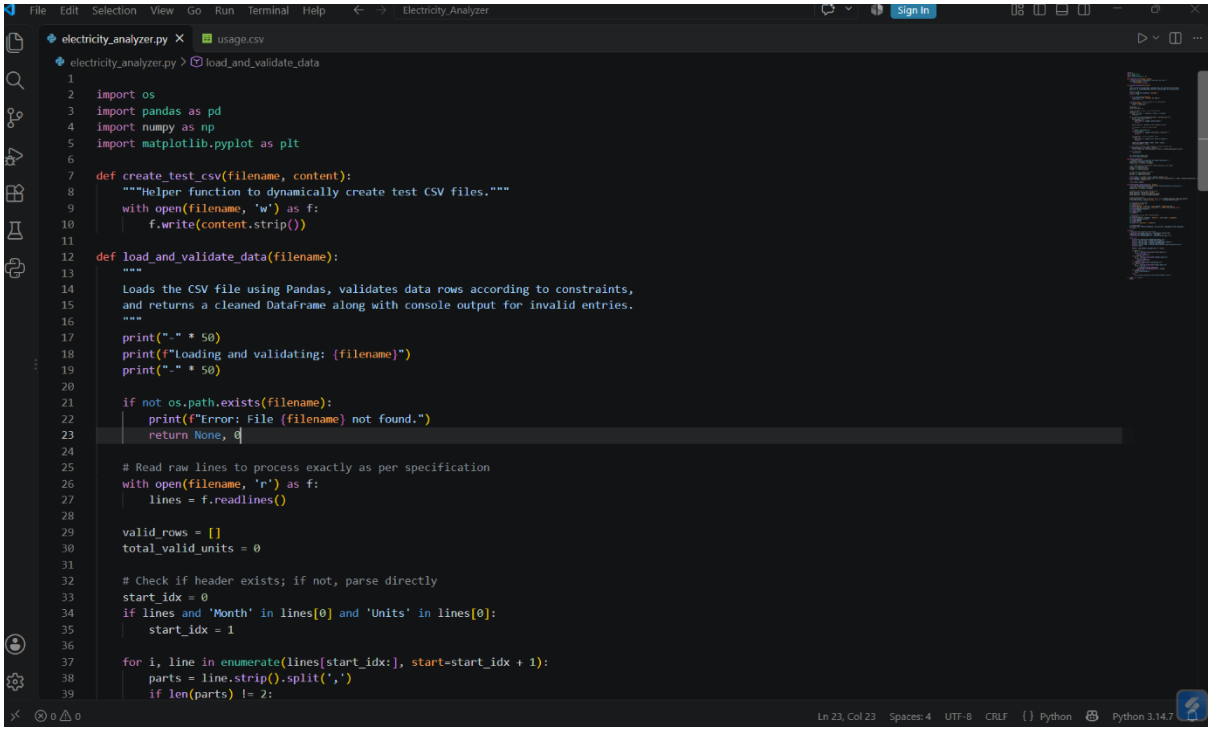


```python
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def create_test_csv(filename, content):
    """Helper function to dynamically create test CSV files."""
    with open(filename, 'w') as f:
        f.write(content.strip())

def load_and_validate_data(filename):
    """
    Loads the CSV file using Pandas, validates data rows according to constraints,
    and returns a cleaned DataFrame along with console output for invalid entries.
    """
    print("-" * 50)
    print(f"Loading and validating: {filename}")
    print("-" * 50)

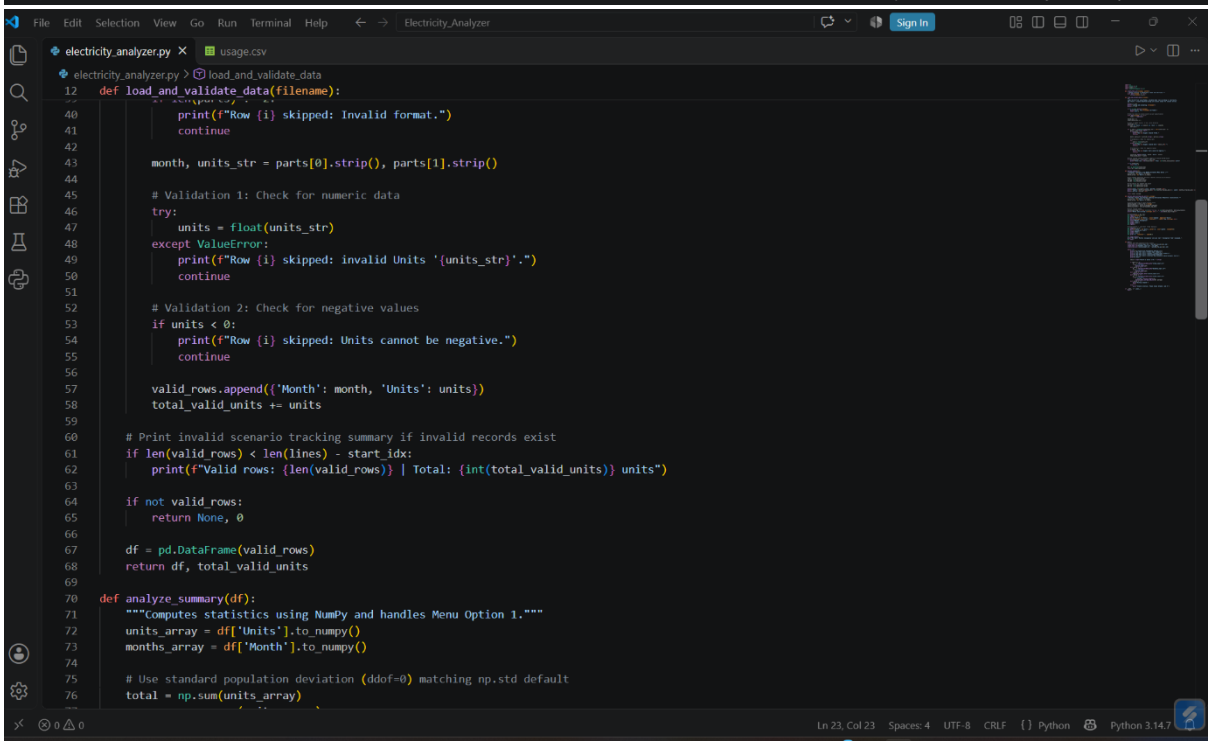
    if not os.path.exists(filename):
        print(f"Error: File {filename} not found.")
        return None, 0

    # Read raw lines to process exactly as per specification
    with open(filename, 'r') as f:
        lines = f.readlines()

    valid_rows = []
    total_valid_units = 0

    # Check if header exists; if not, parse directly
    start_idx = 0
    if lines and 'Month' in lines[0] and 'Units' in lines[0]:
        start_idx = 1

    for i, line in enumerate(lines[start_idx:], start=start_idx + 1):
        parts = line.strip().split(',')
        if len(parts) != 2:
```



```python
def load_and_validate_data(filename):
            print(f"Row {i} skipped: Invalid format.")
            continue

        month, units_str = parts[0].strip(), parts[1].strip()

        # Validation 1: Check for numeric data
        try:
            units = float(units_str)
        except ValueError:
            print(f"Row {i} skipped: invalid Units '{units_str}'.")
            continue

        # Validation 2: Check for negative values
        if units < 0:
            print(f"Row {i} skipped: Units cannot be negative.")
            continue

        valid_rows.append({'Month': month, 'Units': units})
        total_valid_units += units

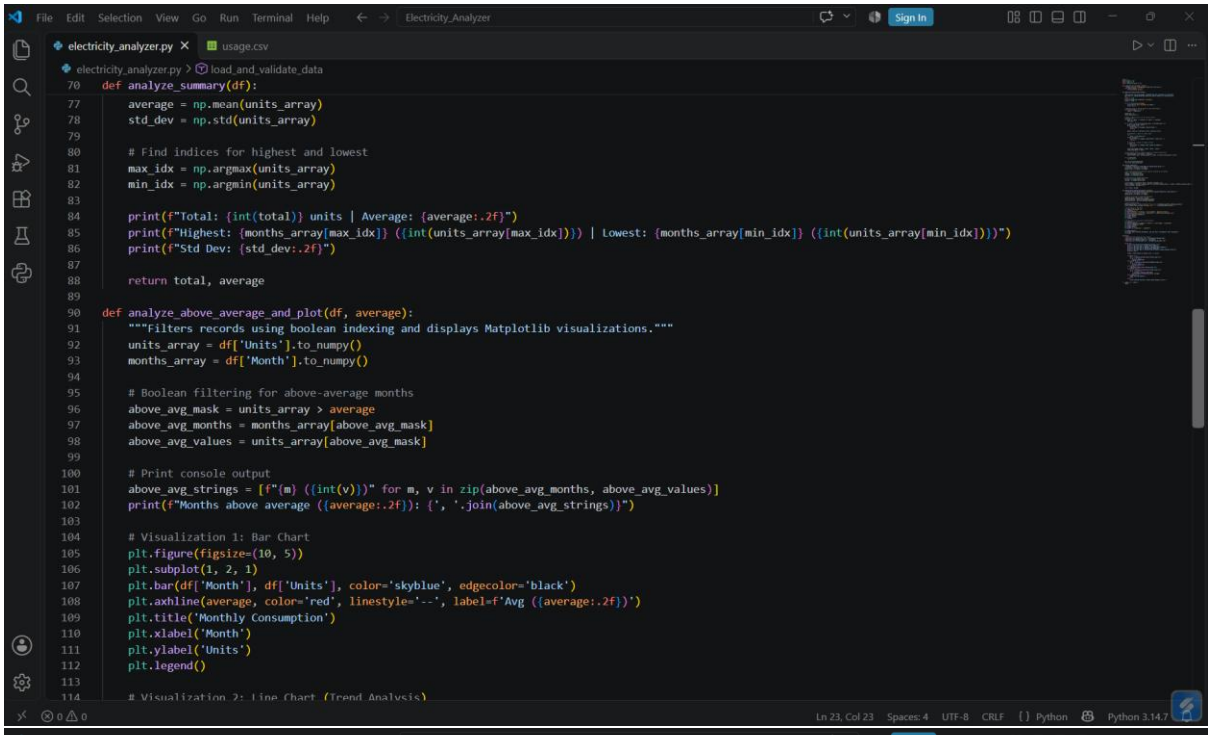
    # Print invalid scenario tracking summary if invalid records exist
    if len(valid_rows) < len(lines) - start_idx:
        print(f"Valid rows: {len(valid_rows)} | Total: {int(total_valid_units)} units")

    if not valid_rows:
        return None, 0

    df = pd.DataFrame(valid_rows)
    return df, total_valid_units

def analyze_summary(df):
    """Computes statistics using NumPy and handles Menu Option 1."""
    units_array = df['Units'].to_numpy()
    months_array = df['Month'].to_numpy()

    # Use standard population deviation (ddof=0) matching np.std default
    total = np.sum(units_array)
```



```
def analyze_summary(df):
    average = np.mean(units_array)
    std_dev = np.std(units_array)

    # Find indices for highest and lowest
    max_idx = np.argmax(units_array)
    min_idx = np.argmin(units_array)

    print(f"Total: {int(total)} units | Average: {average:.2f}")
    print(f"Highest: {months_array[max_idx]} ({int(units_array[max_idx])}) | Lowest: {months_array[min_idx]} ({int(units_array[min_idx])})")
    print(f"Std Dev: {std_dev:.2f}")

    return total, average

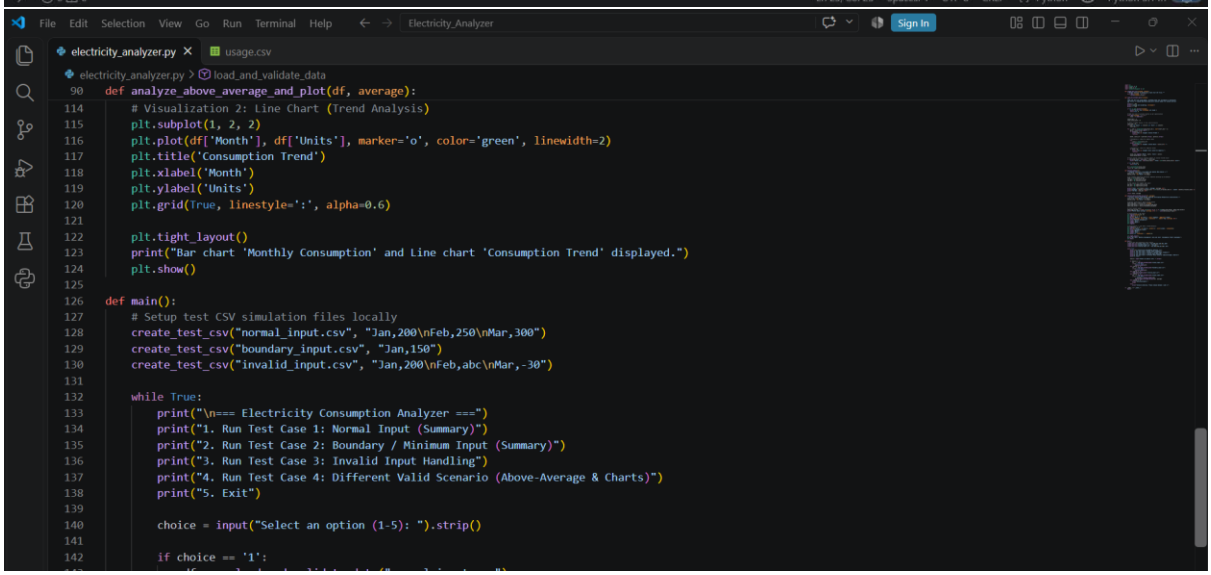
def analyze_above_average_and_plot(df, average):
    """Filters records using boolean indexing and displays Matplotlib visualizations."""
    units_array = df['Units'].to_numpy()
    months_array = df['Month'].to_numpy()

    # Boolean filtering for above-average months
    above_avg_mask = units_array > average
    above_avg_months = months_array[above_avg_mask]
    above_avg_values = units_array[above_avg_mask]

    # Print console output
    above_avg_strings = [f"{m} ({int(v)})" for m, v in zip(above_avg_months, above_avg_values)]
    print(f"Months above average ({average:.2f}): {', '.join(above_avg_strings)}")

    # Visualization 1: Bar Chart
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.bar(df['Month'], df['Units'], color='skyblue', edgecolor='black')
    plt.axhline(average, color='red', linestyle='--', label=f'Avg ({average:.2f})')
    plt.title('Monthly Consumption')
    plt.xlabel('Month')
    plt.ylabel('Units')
    plt.legend()

    # Visualization 2: Line Chart (Trend Analysis)
```



```
def analyze_above_average_and_plot(df, average):
    # Visualization 2: Line Chart (Trend Analysis)
    plt.subplot(1, 2, 2)
    plt.plot(df['Month'], df['Units'], marker='o', color='green', linewidth=2)
    plt.title('Consumption Trend')
    plt.xlabel('Month')
    plt.ylabel('Units')
    plt.grid(True, linestyle=':', alpha=0.6)

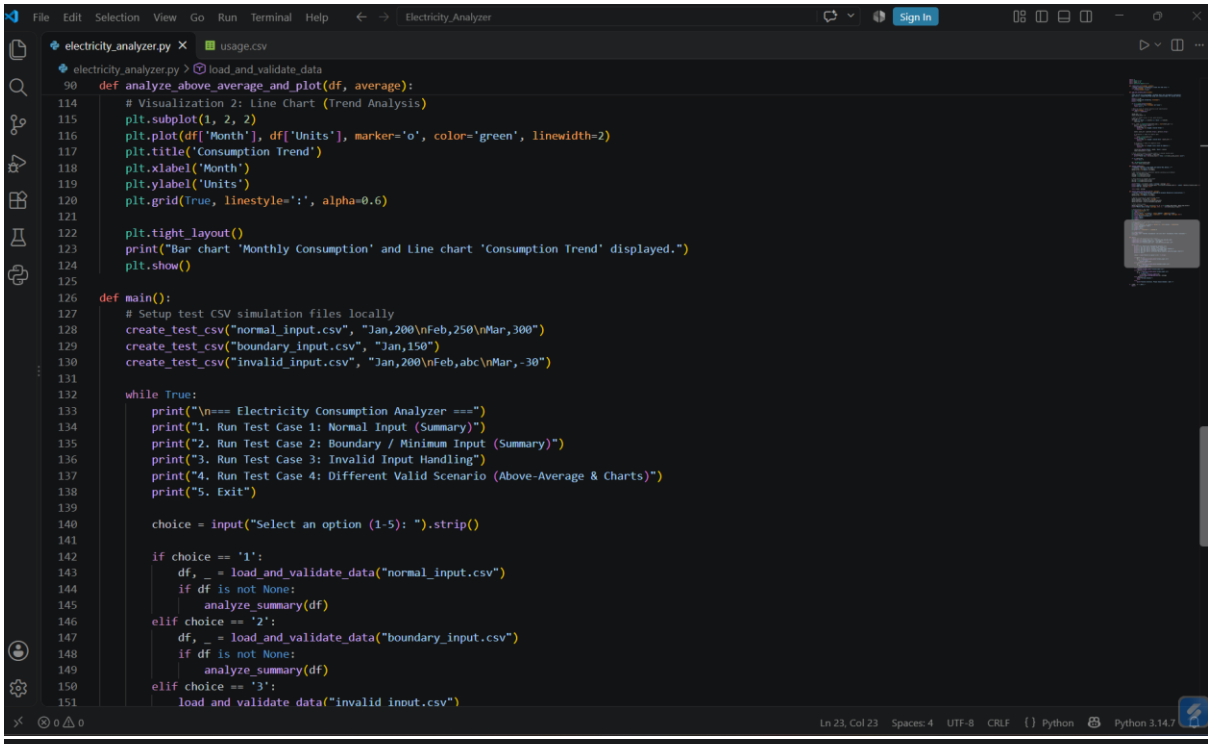
    plt.tight_layout()
    print("Bar chart 'Monthly Consumption' and Line chart 'Consumption Trend' displayed.")
    plt.show()

def main():
    # Setup test CSV simulation files locally
    create_test_csv("normal_input.csv", "Jan,200\nFeb,250\nMar,300")
    create_test_csv("boundary_input.csv", "Jan,150")
    create_test_csv("invalid_input.csv", "Jan,200\nFeb,abc\nMar,-30")

    while True:
        print("\n=== Electricity Consumption Analyzer ===")
        print("1. Run Test Case 1: Normal Input (Summary)")
        print("2. Run Test Case 2: Boundary / Minimum Input (Summary)")
        print("3. Run Test Case 3: Invalid Input Handling")
        print("4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)")
        print("5. Exit")

        choice = input("Select an option (1-5): ").strip()

        if choice == '1':
```



```
def analyze_above_average_and_plot(df, average):
    # Visualization 2: Line Chart (Trend Analysis)
    plt.subplot(1, 2, 2)
    plt.plot(df['Month'], df['Units'], marker='o', color='green', linewidth=2)
    plt.title('Consumption Trend')
    plt.xlabel('Month')
    plt.ylabel('Units')
    plt.grid(True, linestyle=':', alpha=0.6)

    plt.tight_layout()
    print("Bar chart 'Monthly Consumption' and Line chart 'Consumption Trend' displayed.")
    plt.show()

def main():
    # Setup test CSV simulation files locally
    create_test_csv("normal_input.csv", "Jan,200\nFeb,250\nMar,300")
    create_test_csv("boundary_input.csv", "Jan,150")
    create_test_csv("invalid_input.csv", "Jan,200\nFeb,abc\nMar,-30")

    while True:
        print("\n=== Electricity Consumption Analyzer ===")
        print("1. Run Test Case 1: Normal Input (Summary)")
        print("2. Run Test Case 2: Boundary / Minimum Input (Summary)")
        print("3. Run Test Case 3: Invalid Input Handling")
        print("4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)")
        print("5. Exit")

        choice = input("Select an option (1-5): ").strip()

        if choice == '1':
            df, _ = load_and_validate_data("normal_input.csv")
            if df is not None:
                analyze_summary(df)
        elif choice == '2':
            df, _ = load_and_validate_data("boundary_input.csv")
            if df is not None:
                analyze_summary(df)
        elif choice == '3':
            load_and_validate_data("invalid_input.csv")
        elif choice == '4':
            df, _ = load_and_validate_data("normal_input.csv")
            if df is not None:
                _, average = analyze_summary(df)
                analyze_above_average_and_plot(df, average)
        elif choice == '5':
            print("Exiting program.")
            break
        else:
            print("Invalid selection. Please choose between 1 and 5.")

if __name__ == "__main__":
    main()
```

File Edit Selection View Go Run Terminal Help Electricity_Analyzer Sign In

electricity_analyzer.py usage.csv

usage.csv > data

```
#testcase1
Month,Units
Jan,200
Feb,250
Mar,300
#testcase2
Jan,150
#testcase3
Jan,200
Feb,abc
Mar,-30
create_test_csv("normal_input.csv", "Jan,200\nFeb,250\nMar,300")
    create_test_csv("boundary_input.csv", "Jan,150")
    create_test_csv("invalid_input.csv", "Jan,200\nFeb,abc\nMar,-30")
```

File Edit Selection View Go Run Terminal Help Electricity_Analyzer Sign In

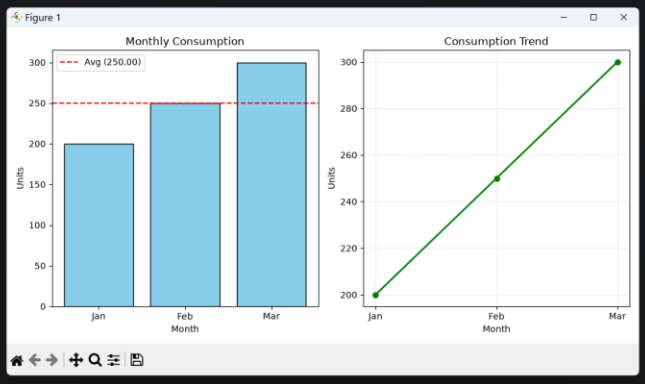
Explorer (Ctrl+Shift+E) Debug Console Terminal Ports Python

```
PS C:\Users\Abhinav Thakur\Desktop\Electricity_Analyzer> & "C:\Users\Abhinav Thakur\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:/Users/Abhinav Thakur/Desktop/Electricity_Analyzer/electricity_analyzer.py"

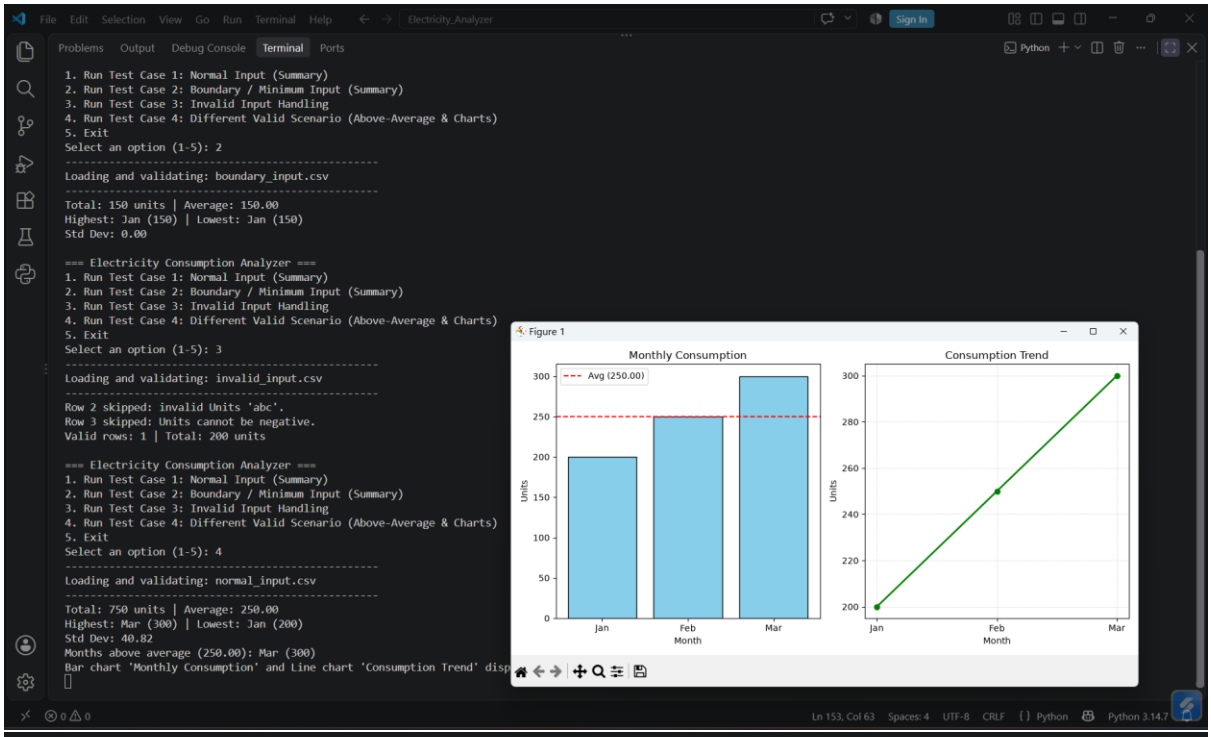
=== Electricity Consumption Analyzer ===
1. Run Test Case 1: Normal Input (Summary)
2. Run Test Case 2: Boundary / Minimum Input (Summary)
3. Run Test Case 3: Invalid Input Handling
4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)
5. Exit
Select an option (1-5): 1
------------------------------------------------------
Loading and validating: normal_input.csv
------------------------------------------------------
Total: 750 units | Average: 250.00
Highest: Mar (300) | Lowest: Jan (200)
Std Dev: 40.82

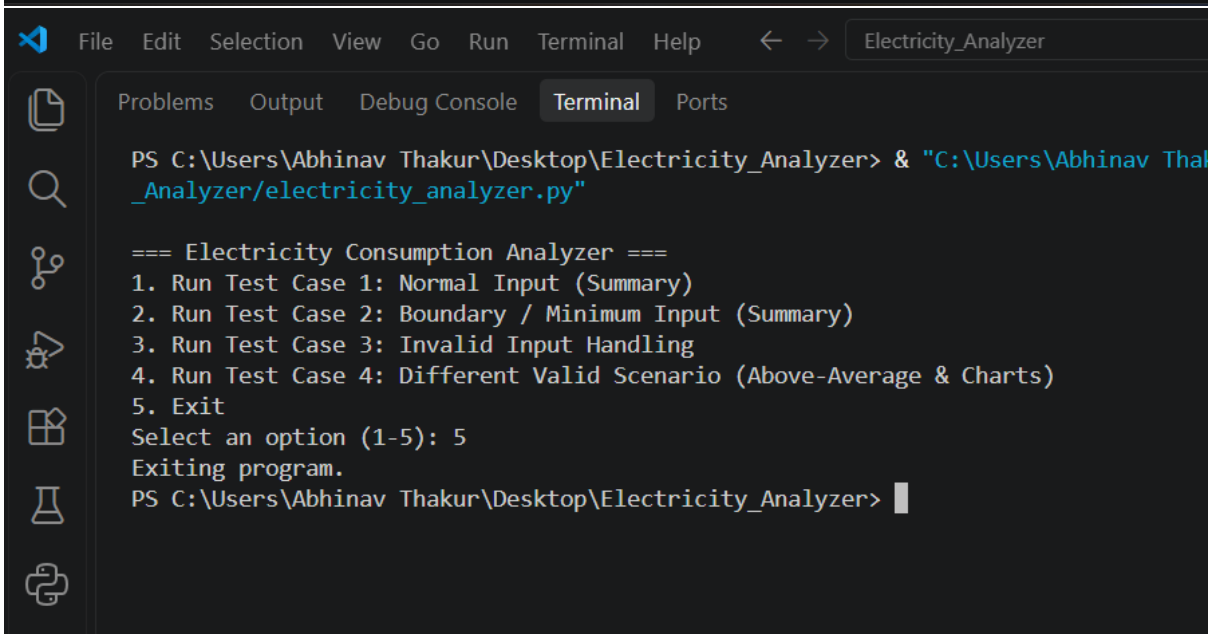
=== Electricity Consumption Analyzer ===
1. Run Test Case 1: Normal Input (Summary)
2. Run Test Case 2: Boundary / Minimum Input (Summary)
3. Run Test Case 3: Invalid Input Handling
4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)
5. Exit
Select an option (1-5): 2
------------------------------------------------------
Loading and validating: boundary_input.csv
------------------------------------------------------
Total: 150 units | Average: 150.00
Highest: Jan (150) | Lowest: Jan (150)
Std Dev: 0.00

=== Electricity Consumption Analyzer ===
1. Run Test Case 1: Normal Input (Summary)
2. Run Test Case 2: Boundary / Minimum Input (Summary)
3. Run Test Case 3: Invalid Input Handling
4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)
5. Exit
Select an option (1-5): 3
------------------------------------------------------
Loading and validating: invalid_input.csv
------------------------------------------------------
Row 2 skipped: invalid Units 'abc'.
Row 3 skipped: Units cannot be negative.
```



Ln 153, Col 63 Spaces: 4 UTF-8 CRLF { } Python Python 3.14.7


File Edit Selection View Go Run Terminal Help
Electricity_Analyzer
Sign In
Problems Output Debug Console Terminal Ports
Python
1. Run Test Case 1: Normal Input (Summary)
2. Run Test Case 2: Boundary / Minimum Input (Summary)
3. Run Test Case 3: Invalid Input Handling
4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)
5. Exit
Select an option (1-5): 2
Loading and validating: boundary_input.csv
Total: 150 units | Average: 150.00
Highest: Jan (150) | Lowest: Jan (150)
Std Dev: 0.00
=== Electricity Consumption Analyzer ===
1. Run Test Case 1: Normal Input (Summary)
2. Run Test Case 2: Boundary / Minimum Input (Summary)
3. Run Test Case 3: Invalid Input Handling
4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)
5. Exit
Select an option (1-5): 3
Loading and validating: invalid_input.csv
Row 2 skipped: invalid Units 'abc'.
Row 3 skipped: Units cannot be negative.
Valid rows: 1 | Total: 200 units
=== Electricity Consumption Analyzer ===
1. Run Test Case 1: Normal Input (Summary)
2. Run Test Case 2: Boundary / Minimum Input (Summary)
3. Run Test Case 3: Invalid Input Handling
4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)
5. Exit
Select an option (1-5): 4
Loading and validating: normal_input.csv
Total: 750 units | Average: 250.00
Highest: Mar (300) | Lowest: Jan (200)
Std Dev: 40.82
Months above average (250.00): Mar (300)
Bar chart 'Monthly Consumption' and Line chart 'Consumption Trend' disp
Figure 1
Monthly Consumption
Avg (250.00)
Units
Month
Jan
Feb
Mar
Consumption Trend
Ln 153, Col 63 Spaces: 4 UTF-8 CRLF Python Python 3.14.7


File Edit Selection View Go Run Terminal Help
Electricity_Analyzer
Problems Output Debug Console Terminal Ports
PS C:\Users\Abhinav Thakur\Desktop\Electricity_Analyzer> & "C:\Users\Abhinav Thak
_Analyzer/electricity_analyzer.py"
=== Electricity Consumption Analyzer ===
1. Run Test Case 1: Normal Input (Summary)
2. Run Test Case 2: Boundary / Minimum Input (Summary)
3. Run Test Case 3: Invalid Input Handling
4. Run Test Case 4: Different Valid Scenario (Above-Average & Charts)
5. Exit
Select an option (1-5): 5
Exiting program.
PS C:\Users\Abhinav Thakur\Desktop\Electricity_Analyzer>